

PÍLDORA FORMATIVA

Spring Data JPA Consultas complejas sin usar **@Query**

Recorrido de la sesión

01 ¿Qué es Spring Data JPA?

02 El problema de abusar de @Query

03 Query Methods

04 Consultas avanzadas con nombres de métodos

05 Specifications

06 Demo en vivo (live coding)

07 When prefer Specifications?

08 When should we still use @Query?

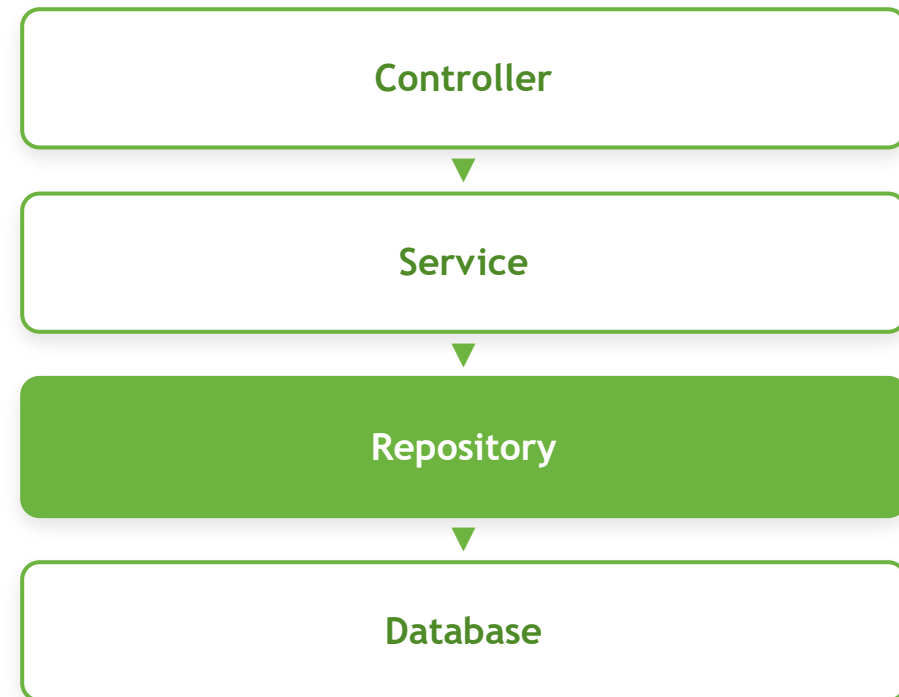
¿Qué es Spring Data JPA?

- Capa de abstracción construida sobre JPA.
- Reduce el código repetitivo (boilerplate).
- Genera las consultas automáticamente a partir de interfaces.

Patrón Repository

Defines una interfaz; Spring provee la implementación en tiempo de ejecución.

Flujo de una petición



El problema de @Query

VENTAJAS

- Muy flexible.
- Permite JPQL y SQL nativo.

DESVENTAJAS

- Difícil mantenimiento.
- Consultas largas.
- Duplicación de lógica.
- Menor legibilidad.

```
@Query(
    "SELECT a FROM Article a " +
    "WHERE a.status = :status " +
    "AND a.category = :cat " +
    "AND LOWER(a.author.name) " +
    "LIKE %:name% " +
    "ORDER BY a.publishedAt DESC"
)
List<Article> search(...);

// Difícil de leer y mantener
```

Consultas a partir del nombre del método

- Spring analiza el nombre del método y genera la consulta.
- Convención: prefijo findBy + propiedad de la entidad.
- Cero implementación, cero SQL escrito a mano.

Regla mental: findBy + Campo → WHERE campo = ?

```
public interface ArticleRepository
    extends JpaRepository<Article, Long> {

    List<Article> findByTitle(String title);
    List<Article> findByAuthor(String author);
    List<Article> findByStatus(Status status);
    List<Article> findByCategory(String cat);
}
```

Consultas más complejas por nombre

findByTitleContaining()

Búsqueda parcial · LIKE %texto%

findByStatusAndCategory()

Dos condiciones combinadas · AND

findByPublishedAtAfter()

Comparación de fechas · > fecha

findByAuthorNameIgnoreCase()

Navega relaciones e ignora mayúsculas

Spring interpreta cada palabra clave: **Containing** · **And** · **After** · **IgnoreCase** se traducen a la cláusula SQL correspondiente.

Palabras clave más útiles

Keyword	Significado (SQL equivalente)
Containing	LIKE %texto%
StartingWith	Empieza por...
EndingWith	Termina por...
And	AND
Or	OR
Between	BETWEEN
LessThan	<
GreaterThan	>
OrderBy	ORDER BY

Cómo leerlo

- Cada keyword equivale a una cláusula SQL.
- Se combinan dentro del nombre del método.
- Ideal para memorizar el vocabulario básico.

¿Dónde empiezan los problemas?

Cuando los filtros crecen, el nombre del método se vuelve interminable:

```
List<Article>  
findByStatusAndCategoryAndAuthorNameContaining  
    AndPublishedAtAfterOrderByPublishedAtDesc(...);
```

✓ Técnicamente funciona

Spring lo interpreta y genera la consulta correcta.

✗ Pero es inmanejable

Ilegible, frágil y no escala si añades otro filtro.

Una solución más elegante

- Consultas dinámicas: se construyen en tiempo de ejecución.
- Filtros combinables como piezas de Lego.
- Más reutilizables: cada filtro vive por separado.

Un mismo conjunto de filtros cubre infinitas combinaciones.

Filtros combinables

status

category

author

date

.and()
.or()



query()

¿Cómo funciona una Specification?

```
Specification<Article> spec = Specification.allOf();  
if (status != null)  
    spec = spec.and(hasStatus(status));  
if (category != null)  
    spec = spec.and(hasCategory(category));  
if (author != null)  
    spec = spec.and(hasAuthor(author));  
  
return repository.findAll(spec);
```

Anatomía

Filtros

status · category · author

Combinación

allOf() + and() condicional

Ejecución

repository.findAll(spec)

API de artículos con filtros opcionales

Escenario

Un endpoint de búsqueda de artículos donde el usuario puede aplicar cualquier combinación de filtros (o ninguno):

Filtros opcionales

Estado

Categoría

Autor

Fecha

Se aplican de forma independiente: uno, varios, todos o ninguno.



¿Cómo evitar crear 20 métodos distintos para cubrir todas las combinaciones?

$2^4 = 16$ combinaciones posibles solo con 4 filtros opcionales

• EN DIRECTO

Live Coding Demo

1 Crear la entidad Article

2 Crear el Repository

3 Extender JpaSpecificationExecutor

4 Crear las Specifications

5 Construir los filtros dinámicos

6 Probar el endpoint

Why prefer Specifications?

✓ **Better maintainability** Fewer long, brittle query strings to babysit.

✓ **Easier refactoring** Rename fields without rewriting SQL by hand.

✓ **Cleaner repositories** Short interfaces, clear intent.

✓ **Reusable filters** Compose criteria across many queries.

✓ **Dynamic search** Build queries from the user's actual input.

When should we still use @Query?

The ladder: start with query methods, escalate to Specifications as filters grow — and keep @Query for when:

The query is extremely specific

A one-off that no generic filter models well.

Complex joins are needed

Multiple tables and fine-grained relationships.

Native SQL is required

Database-specific features or functions.

Performance tuning demands full control

Hand-optimized queries and hints.

GRACIAS

¿Preguntas?

¡Gracias por vuestra atención!

Spring Data JPA · Consultas complejas sin usar @Query

